

操作系统实验报告

姓名：李宗杭 学号：2311451 学院：软件学院

实验日期：2025.11.12 星期三 11-12 节

实验题目：破解秘密消息

一、实验内容

通过传入正确的 4 个运行时参数 (key1-key4)，不修改 mysecret.cpp 源代码，从 data 数组中提取出隐藏的秘密消息。

二、实现步骤

1、环境准备

安装/更新核心编译工具：sudo apt-get install -y build-essential

2、原理分析

(1) 根据作业中提供的参考文档思路，破解过程可以分为两个阶段：先破解 key1 和 key2，触发以 "From: " 开头的伪装消息；再破解 key3 和 key4，强制程序调用 extract_message2 而非 extract_message1，提取真实秘密消息。

(2) 关键变量：

dummy：初始值 1，被 process_keys12 修改后，其第 0 字节作为 start，第 1 字节作为 stride；

start：消息提取起始偏移量；

stride：消息提取步长；

data[]：存储加密后的消息（十六进制），需通过 start 和 stride 过滤无效数据；

key1-key4：破解目标。

(3) 关键函数：

process_keys12()：通过指针操作修改 dummy 的值，间接控制 start 和 stride；

process_keys34()：通过修改函数栈帧返回地址，让程序跳过 extract_message1 直接执行 extract_message2；

`extract_message1()`: 按 `start+1` 起始、每隔一个字节取 `stride-1` 字节;

`extract_message2()`: 按 `start` 起始、每隔 `stride` 字节取一个字节。

(4) 32 位程序中, 函数调用遵循 `cdecl` 约定, 栈帧包含参数、返回地址、旧 `ebp` 等关键信息, 栈内存向下生长, 局部变量在栈中连续分配, 通过精准计算地址偏移可修改返回地址。

3、源文件编译

将 `mysecret.cpp` 编译为 32 位带调试信息的可执行文件:

```
gcc -m32 -g -o mysecret mysecret.cpp
```

4、破解 key1、key2

(1) 分析 `process_keys12` 逻辑

`key1` 是密钥 1 的指针, `*key1` 是密钥 1 的数值, 计算一个新地址 `key1 + *key1`, 然后把密钥 2 的数值 `*key2` 写入这个地址。

从主函数中可以看到在执行 `process_keys12` 后, 便要通过 `dummy` 获取 `start` 和 `stride`, 随后便直接执行 `extract_message1()` 获取诱饵消息。因此 `dummy` 只能是通过 `process_keys12` 修改, 所以 `process_keys12` 的任务便是通过 `key1` 找到 `dummy` 的内存地址, 然后把正确的值通过 `key2` 写入该地址。

(2) 破解 key1

破解 `key1` 便是要找到 `&key` 与 `&dummy` 之间的关系。在 `process_keys12` 中, `key1` 是指针 (`int*`), 新地址为 `((int*)(key1 + *key1))`, 也就是 `&dummy = key1 + *key1`, 在 32 位系统中, `int` 指针加 `n` 等于地址加 `n*4` 字节, 因此 `key1(整型 int) = (&dummy - &key1) / 4`。

启用 GDB 调试, 在 `main` 处打断点, 运行程序并随便传入两个密钥值, 然后查看 `key1` 和 `dummy` 的地址。如图得到 `&key1 = 0xffffffd4`, `&dummy = 0xffffffd0`, 根据刚才分析出的式子便可算出 `key1 = -1`。

```
LZH2311451@LZH2311451-pc:~$ gdb ./mysecret
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./mysecret...
(gdb) b main
Breakpoint 1 at 0x80493cf: file mysecret.cpp, line 90.
(gdb) r 1 2
Starting program: /home/LZH2311451/mysecret 1 2

Breakpoint 1, main (argc=3,
    argv=<error reading variable: Cannot access memory at address 0xffffffc4>)
    at mysecret.cpp:90
90      {
(gdb) p &key1
$1 = (int *) 0xffffffd4
(gdb) p &dummy
$2 = (int *) 0xffffffd0
```

(3) 破解 key2

```
start = (int)*(((char *) &dummy));  
stride = (int)*(((char *) &dummy) + 1));
```

根据如上代码，且 int 占 4 字节、char 占 1 字节，可以得知 start 和 stride 的取值逻辑是将 int* 类型指针 &dummy 强转为 char* 后按单字节读取。而由于系统采取小端存储（低字节存低地址），假设原 int *dummy 地址为 0xffffffffd0、值为 0x12345678 的话，字节地址应为：*(0xffffffffd0)=0x78，*(0xffffffffd1)=0x56，*(0xffffffffd2)=0x34，*(0xffffffffd3)=0x12。因此，应使 78 位置为 start 的值，56 位置为 stride 的值。

接下来分析 start 和 stride 具体值。观察 extract_message1() 函数的取数逻辑，发现其以一个变量 j 作为偏移量，来获取相对于 data[] 首地址偏移 j 后的字节，起始偏移量为 start + 1。内存循环执行 stride - 1 次，每次将偏移量 j 加 1，外层循环也会对 j 加 1，也就是每隔 1 字节取 stride - 1 字节。

另外题目中提供了未加密消息的前五个字符 "From:"，于是可以在 data 数组中直接观察：是否可以通过每隔一个字节取连续 stride 字节的方式得到 "From:"。

可以在 gdb 中查看按字节地址顺序排列的 16 进制数与其对应的 ASCII 编码字符。发现从第 11 个字节开始取，每次取 2 个字节，便能得到 "From:"，因此起始偏移量为 10，则 start 为 9，stride 为 3。因此 dummy 应为 0x00000309，即 key2 = 0x00000309 = 777。

```
(gdb) x/128xb data  
0x804c0e0 <data>: 0x63 0x63 0x63 0x63 0x63 0x63 0x63 0x63 0x63  
0x804c0e8 <data+8>: 0x63 0x46 0x46 0x72 0x72 0x6f 0x6d 0x6f  
0x804c0f0 <data+16>: 0x3a 0x20 0x6d 0x46 0x72 0x3a 0x69 0x65  
0x804c0f8 <data+24>: 0x20 0x6e 0x64 0x43 0x0a 0x54 0x54 0x6f  
0x804c100 <data+32>: 0x3a 0x45 0x20 0x59 0x0a 0x6f 0x75 0x54  
0x804c108 <data+40>: 0x0a 0x47 0x6f 0x6f 0x6f 0x3a 0x64 0x21  
0x804c110 <data+48>: 0x20 0x20 0x4e 0x59 0x6f 0x77 0x6f 0x20  
0x804c118 <data+56>: 0x74 0x75 0x72 0x79 0x0a 0x20 0x63 0x45  
0x804c120 <data+64>: 0x68 0x6f 0x78 0x6f 0x73 0x63 0x69 0x6e  
0x804c128 <data+72>: 0x65 0x67 0x20 0x6c 0x6b 0x65 0x6c 0x79  
0x804c130 <data+80>: 0x73 0x65 0x33 0x2c 0x6e 0x34 0x20 0x74  
0x804c138 <data+88>: 0x74 0x6f 0x21 0x20 0x66 0x59 0x6f 0x72  
0x804c140 <data+96>: 0x6f 0x63 0x65 0x75 0x20 0x61 0x20 0x20  
0x804c148 <data+104>: 0x63 0x67 0x61 0x6c 0x6f 0x6c 0x20 0x74  
0x804c150 <data+112>: 0x74 0x6f 0x20 0x20 0x65 0x65 0x78 0x74  
0x804c158 <data+120>: 0x76 0x72 0x61 0x65 0x63 0x54 0x72 0x32  
(gdb) x/128c data  
0x804c0e0 <data>: 99 'c' 99 'c' 99 'c' 99 'c' 99 'c' 99 'c' 99 'c' 99 'c'  
0x804c0e8 <data+8>: 99 'c' 70 'F' 70 'F' 114 'r' 114 'r' 111 'o' 109 'm' 111 'o'  
0x804c0f0 <data+16>: 58 ': ' 32 ' ' 109 'm' 70 'F' 114 'r' 58 ': ' 105 'i' 101 'e'  
0x804c0f8 <data+24>: 32 ' ' 110 'n' 100 'd' 67 'C' 10 '\n' 84 'T' 84 'T' 111 'o'  
0x804c100 <data+32>: 58 ': ' 69 'E' 32 ' ' 89 'Y' 10 '\n' 111 'o' 117 'u' 84 'T'  
0x804c108 <data+40>: 10 '\n' 71 'G' 111 'o' 111 'o' 111 'o' 58 ': ' 100 'd' 33 '!'  
0x804c110 <data+48>: 32 ' ' 32 ' ' 78 'N' 89 'Y' 111 'o' 119 'w' 111 'o' 32 ' '  
0x804c118 <data+56>: 116 't' 117 'u' 114 'r' 121 'y' 10 '\n' 32 ' ' 99 'c' 69 'E'  
0x804c120 <data+64>: 104 'h' 111 'o' 120 'x' 111 'o' 115 's' 99 'c' 105 'i' 110 'n'  
0x804c128 <data+72>: 101 'e' 103 'g' 32 ' ' 108 'l' 107 'k' 101 'e' 108 'l' 121 'y'  
0x804c130 <data+80>: 115 's' 101 'e' 51 '3' 44 ',' 110 'n' 52 '4' 32 ' ' 116 't'  
0x804c138 <data+88>: 116 't' 111 'o' 33 '!' 32 ' ' 102 'f' 89 'Y' 111 'o' 114 'r'  
0x804c140 <data+96>: 111 'o' 99 'c' 101 'e' 117 'u' 32 ' ' 97 'a' 32 ' ' 32 ' '  
0x804c148 <data+104>: 99 'c' 103 'g' 97 'a' 108 'l' 111 'o' 108 'l' 32 ' ' 116 't'  
0x804c150 <data+112>: 116 't' 111 'o' 32 ' ' 32 ' ' 101 'e' 101 'e' 120 'x' 116 't'  
0x804c158 <data+120>: 118 'v' 114 'r' 97 'a' 101 'e' 99 'c' 116 't' 114 'r' 50 '2'
```

(4) 验证 key1、key2

将刚才得到的密钥 key1 = -1、key2 = 777 作为参数运行程序。可以看到成功得到了诱饵信息。

```
LZH2311451@LZH2311451-pc:~$ ./mysecret -1 777
From: Friend
To: You
Good! Now try choosing keys3,4 to force a call to extract2 and
avoid the call to extract1
Press any key to exit...
```

5、破解 key3、key4

(1) 分析现状

按照题目，和第一部分输出“to force a call to extract2 and avoid the call to extract1”，需要在解决前两个密钥的基础上，强制使程序调用 extract_message2 并避免调用 extract_message1。在主函数中，唯一还有操作空间的便是在 extract_message1 之前调用的 process_keys34。

(2) 分析 process_keys34 逻辑

&key3 是密钥 3 的指针本身的地址（指针变量也在内存栈中），把它转成 int* 指针后，加上 *key3 得到新地址，给这个地址的变量加上 *key4。可以大概看出，process_keys34 的意图是修改 &key3 附近某个地址的值。

代码中的显式逻辑是要想执行 extract_message2，则要满足 *msg1 == '\0'，msg1 是 extract_message1 根据 start 和 stride 在 data[] 读取的消息，而 process_keys34 在 extract_message1 之前且在为 start 和 stride 赋值之后被调用，看起来似乎可以通过 process_keys34 来修改某些值使得 *msg1 == '\0' 成立（比如让 start 变为 data[] 中某些 0x00 的位置，或者将 data[] 中 start 位置的值改为 0x00），但刚才已经分析过 process_keys34 的意图是修改 &key3 附近某个地址的值，而 &start 和 data 相对 &key3 的偏移量都十分大，显然不合理。并且如此做不满足题目中“避免调用 extract_message1”的要求。

题目中给出了提示“每个函数调用的活动记录（栈帧）都集中在内存中，且以栈的形式组织：函数调用时栈向下增长，函数返回时栈向上收缩。一个函数的返回地址及其所有局部变量的地址，都分配在同一个活动记录中。”，那么就可以查看一下栈帧中到底有什么。

通过 info frame 命令获取栈帧信息，下表中展示了主要信息项。

信息	值	说明
Stack level 0	0xffffd030	函数当前栈帧的顶部地址
saved eip	0x804950e	保存的返回地址，是 process_keys34 执行完成后，默认要返回的 main 函数地址
args: key3	0xffffd064	main 函数中 key3 变量的地址 (传入 process_keys34 的参数)
eip at	0xffffd02c	saved eip 的内存存储地址

```

LZH2311451@LZH2311451-pc:~$ gdb ./mysecret
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./mysecret...
(gdb) b process_keys34
Breakpoint 1 at 0x80492ba: file mysecret.cpp, line 52.
(gdb) r -1 777 3 4
Starting program: /home/LZH2311451/mysecret -1 777 3 4

Breakpoint 1, process_keys34 (key3=0xffffd064, key4=0xffffd068) at mysecret.cpp:52
52 void process_keys34 (int * key3, int * key4) {
(gdb) info frame
Stack level 0, frame at 0xffffd030:
eip = 0x80492ba in process_keys34 (mysecret.cpp:52); saved eip = 0x804950e
called by frame at 0xffffd0a0
source language c++.
Arglist at 0xffffd028, args: key3=0xffffd064, key4=0xffffd068
Locals at 0xffffd028, Previous frame's sp is 0xffffd030
Saved registers:
eip at 0xffffd02c

```

内容并不多，但已足够，有了“eip at”（也就是 saved eip）的地址后，便可对 saved eip（函数返回地址）进行修改，从而实现题目中“强制使程序调用 extract_message2 并避免调用 extract_message1”的要求。因此在 process_keys34 中，便可用 key3 来找到 eip 的地址，然后用 key4 修改其值。

（3）破解 key3

破解 key3 便是要找到 &key3 与 eip 之间的关系。类比破解 key1 时的分析，可以得知 $*key3 = (\&eip - \&key3) / 4$ （其中 key3 为 process_keys34 中的指针变量）。

&eip 从栈帧信息中已获取，即 0xffffd02c（“eip at”），&key3 的获取也很简单，如下图，为 0xffffd030，于是，可以算得 $*key3 = -1$ ，即密钥 3 = -1。

```

(gdb) info frame
Stack level 0, frame at 0xffffd030:
eip = 0x80492ba in process_keys34 (mysecret.cpp:52); saved eip = 0x804950e
called by frame at 0xffffd0a0
source language c++.
Arglist at 0xffffd028, args: key3=0xffffd064, key4=0xffffd068
Locals at 0xffffd028, Previous frame's sp is 0xffffd030
Saved registers:
eip at 0xffffd02c
(gdb) p &key3
$1 = (int **) 0xffffd030

```

（4）破解 key4

由于目标是修改返回地址，因此需要找到 *key4 与 eip 之间的关系。根据 process_keys34 中的计算规则 $*(((int *)\&key3) + *key3) += *key4$ ，得知 $*key4 = target_address - eip$ 。eip 即 saved eip，为 0x804950e。

target_address 则是主函数中调用 extract_message2 的指令地址，需要通过 disass main 反编译获得。如下图，在反编译结果中可以找到这部分内容。其中 0x0804954b <+380>: call 0x8049375 <extract_message2(int, int)>便是调用 extract_message2 的指令，但不能直接将其地址作为 target_address 的值，还需要执行其前面的三条参数压栈指令（sub, pushl, pushl），因此

target_address 应为 sub 指令的地址 0x08049542。

```
0x0804952b <+348>: test    %al,%al
0x0804952d <+350>: jne     0x08049566 <main(int, char**)+407>
0x0804952f <+352>: sub     $0x8,%esp
0x08049532 <+355>: lea     -0x20(%ebp),%eax
0x08049535 <+358>: push    %eax
0x08049536 <+359>: lea     -0x24(%ebp),%eax
0x08049539 <+362>: push    %eax
0x0804953a <+363>: call    0x080492ba <process_keys34(int*, int*)>
0x0804953f <+368>: add     $0x10,%esp
0x08049542 <+371>: sub     $0x8,%esp
0x08049545 <+374>: pushl   -0x18(%ebp)
0x08049548 <+377>: pushl   -0x1c(%ebp)
0x0804954b <+380>: call    0x08049375 <extract_message2(int, int)>
0x08049550 <+385>: add     $0x10,%esp
0x08049553 <+388>: mov     %eax,-0x10(%ebp)
0x08049556 <+391>: sub     $0xc,%esp
0x08049559 <+394>: pushl   -0x10(%ebp)
0x0804955c <+397>: call    0x080490f0 <puts@plt>
0x08049561 <+402>: add     $0x10,%esp
```

最终算出的*key4 为十进制 52，即密钥 4 = 52。

(5) 验证 key3、key4

将所有密钥 key1 = -1、key2 = 777、key3 = -1、key4 = 52 作为参数运行程序。可以看到成功得到了秘密消息。

```
LZH2311451@LZH2311451-pc:~$ ./mysecret -1 777 -1 52
From: CTE
To: You
Excellent!You got everything!
Press any key to exit...
```

三、结论

本次实验成功破解了隐藏在 mysecret 程序中的秘密消息，最终确定四个关键参数分别为 key1=-1、key2=777、key3=-1、key4=52，提取出的秘密消息为“From: CTE\nTo: You\nExcellent!You got everything!”。

实验过程完全按照题目中“先破解前两个密钥、再破解后两个密钥”的要求，通过 GDB 调试工具精准获取了变量内存地址、栈帧结构及指令地址等关键信息，利用 32 位程序的内存布局特性、小端存储规则和函数栈帧原理，成功通过指针操作修改目标变量值与函数返回地址，实现了不修改源代码即可提取秘密消息的目标。

此次实验加深了我对函数调用方式、栈帧结构及地址偏移计算的理解，同时加强了基于汇编指令与源代码结合分析问题的能力，受益匪浅。